LAPORAN PRAKTIKUM

STRUKTUR DATA

“Single Linked List”

disusun Oleh:

Sofian Arba’i

2511533029

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kehadirat Allah SWT karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan Laporan Praktikum pekan 5 dengan judul "Single Linked List" ini dengan baik. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum serta sebagai sarana untuk meningkatkan pemahaman mengenai penggunaan struktur data Single Linked List.

Dalam praktikum pekan 5 mempelajari dasar-dasar Single Linked List, yaitu salah satu struktur data linear yang terdiri dari kumpulan node yang saling terhubung. Setiap node pada Single Linked List memiliki data dan pointer yang menunjuk ke node berikutnya. Praktikum ini membahas proses pembuatan Single Linked List, operasi dasar seperti traversal, penambahan node, penghapusan node, dan pencarian data, serta implementasinya dalam pemrograman Java.

Penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, saya ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Dosen pengampu mata kuliah Praktikum Struktur Data yang telah memberikan ilmu, arahan, serta bimbingannya.
2. Asisten praktikum yang telah membimbing dan memberikan penjelasan selama kegiatan praktikum berlangsung.

Saya meminta maaf jika masih terdapat kesalahan pada penulisan laporan ini. Oleh karena itu, saya menerima kritik dan saran yang membangun demi perbaikan di masa mendatang.

Padang, 4 Mei 2026

Sofian Arba'i

DAFTAR ISI

KATA PENGANTAR i
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 2
BAB II PEMBAHASAN 3
2.1 Pengertian Single Linked List 3
2.1.1 Kelebihan dan Kekurangan Single Linked List 3
2.2 Implementasi Single Linked List pada Java 4
2.2.1 Class NodeSLL_2511533029 4
2.2.2 Class TambahSLL_2511533029 5
2.2.3 Class PencarianSLL_2511533029 7
2.2.4 Class HapusSLL_2511533029 9
BAB III PENUTUP 13
3.1 Kesimpulan 13
3.2 Saran 13
DAFTAR PUSTAKA 14

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep dasar dalam pemrograman yang digunakan untuk mengatur, menyimpan, dan mengelola data agar dapat diproses secara efisien. Salah satu bentuk struktur data linear yang penting untuk dipelajari adalah Single Linked List. Single Linked List merupakan struktur data yang terdiri dari kumpulan node, di mana setiap node menyimpan data dan sebuah pointer yang menunjuk ke node berikutnya. Berbeda dengan array yang memiliki ukuran tetap, Single Linked List bersifat dinamis sehingga memudahkan proses penambahan dan penghapusan data.

Pada praktikum pekan 5, mahasiswa mempelajari implementasi Single Linked List menggunakan bahasa pemrograman Java. Praktikum ini bertujuan untuk memahami cara kerja Single Linked List serta berbagai operasi dasar yang dapat dilakukan, seperti traversal, penambahan node, penghapusan node, dan pencarian data. Dengan mempelajari Single Linked List, saya diharapkan mampu memahami salah satu struktur data dasar yang banyak digunakan dalam pengembangan program.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum pekan 5 tentang Single Linked List adalah sebagai berikut:

1. Memahami konsep dasar struktur data Single Linked List.
2. Mempelajari cara membuat dan mengimplementasikan Single Linked List menggunakan Java.
3. Memahami operasi dasar pada Single Linked List seperti traversal, penambahan, penghapusan, dan pencarian node.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum pekan 5 tentang Single Linked List adalah sebagai berikut:

1. Mahasiswa dapat memahami konsep dan cara kerja Single Linked List sebagai struktur data linear.
2. Mahasiswa dapat mengimplementasikan operasi dasar Single Linked List dalam pemrograman Java.
3. Mahasiswa dapat melatih kemampuan logika pemrograman dalam mengelola data secara dinamis.

BAB II
PEMBAHASAN

2.1 Pengertian Single Linked List

Single Linked List adalah salah satu dasar dalam struktur data yang terdiri dari beberapa Node berisi Data dan Pointer ke Node berikutnya, sehingga setiap Node akan saling tehubung menjadi sekumpulan data dimulai dari Node pertama yaitu Head yang berupa data sampai dengan Node terakhir yaitu Tail yang bernilai null.

Ukurannya Dinamis bisa bertambah atau berkurang secara dinamis menyesuaikan dengan kebutuhan Traversal, untuk mencari data tertentu kita harus menelusuri satu-persatu dari Head sampai dengan Tail. Non Indeks, Single linked list tidak bisa diakses langsung menggunakan indeks seperti array karena memiliki struktur penyimpanan data yang berbeda. Array menyimpan data secara berurutan didalam memori tetapi single linked list menyimpan data secara acak dan saling terhubung melalui Pointer.

2.1.1 Kelebihan dan Kekurangan Single Linked List

Meskipun struktur data single linked list memiliki banyak kegunaan, berikut beberapa kelebihan dan kekurangan yang perlu diperhatikan sebelum menggunakannya.

Kelebihan:

1. Struktur sederhana
2. Penggunaan memori lebih hemat
3. Mudah diimplementasikan

Kekurangan:

1. Tidak bisa mundur ke node sebelumnya

2. Proses pencarian harus dimulai dari awal
3. Akses data lebih lambat dibanding array

2.2 Implementasi Single Linked List pada Java

Berikut implementasi penggunaan single linked list pada bahasa pemrograman java sesuai dengan intruksi dosen dan asisten praktikum

2.2.1 Class NodeSLL_2511533029

Kode program pertama merupakan dasar pembentukan struktur data Single Linked List. Kelas ini digunakan untuk membuat sebuah node yang berisi data dan pointer ke node berikutnya. Setiap node menyimpan satu nilai pada variabel data serta memiliki hubungan dengan node lain melalui pointer next.

```java
package pekan5_2511533029;

public class NodeSLL_2511533029 {
    //Node Bagian Data
    int data_3029;
    //Pointer ke node berikutnya
    NodeSLL_2511533029 next_3029;
    // Konsstruktor menginisialisasi node dengan data
    public NodeSLL_2511533029 (int data_3029) {
        this.data_3029 = data_3029;
        this.next_3029 = null;
    }

}
```

Kode Program 2.1

Program NodeSLL_2511533029 merupakan kelas yang digunakan untuk merepresentasikan sebuah node pada struktur data Single Linked List. Di dalam kelas ini terdapat dua atribut utama, yaitu data_3029 yang berfungsi untuk menyimpan nilai atau data pada node, dan next_3029 yang berfungsi sebagai pointer atau penunjuk ke node berikutnya dalam linked list. Atribut next_3029 bertipe NodeSLL_2511533029, sehingga setiap node dapat saling terhubung

dengan node lainnya. Selain itu, kelas ini memiliki sebuah konstruktor NodeSLL_2511533029(int data_3029) yang digunakan untuk menginisialisasi node baru. Konstruktor ini akan mengisi nilai data_3029 sesuai data yang diberikan, lalu mengatur next_3029 bernilai null, yang menandakan bahwa node tersebut belum terhubung ke node lain. Dengan demikian, kelas ini menjadi dasar pembentukan elemen-elemen pada Single Linked List.

2.2.2 Class TambahSLL_2511533029

Kode program selanjutnya yaitu kode program yang menunjukkan bahwa Single Linked List dapat dimodifikasi secara dinamis dengan menambahkan node di berbagai posisi. Melalui implementasi ini, dapat dipahami bahwa operasi penambahan pada Single Linked List dilakukan dengan mengatur hubungan antar node melalui pointer. Berikut kode program tambah SLL:

```java
package pekan5_2511533029;

public class TambahSLL_2511533029 {
    public static NodeSLL_2511533029 insertAtFront(NodeSLL_2511533029 head_3029, int value_3029) {
        NodeSLL_2511533029 new_node_3029 = new NodeSLL_2511533029(value_3029);
        new_node_3029.next_3029 = head_3029;
        return new_node_3029;
    }
    // fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511533029 insertAtEnd(NodeSLL_2511533029 head_3029, int value_3029) {
        // buat sebuah node dengan sebuah nilai
        NodeSLL_2511533029 newNode_3029 = new NodeSLL_2511533029(value_3029);
        // jika list kosong maka node jadi head
        if (head_3029 == null) {
            return newNode_3029;
        }
        // simpan head ke variabel sementara
        NodeSLL_2511533029 last_3029 = head_3029;
        // telusuri ke node akhir
        while (last_3029.next_3029 != null) {
            last_3029 = last_3029.next_3029;
        }
        // ubah pointer
        last_3029.next_3029 = newNode_3029;
        return head_3029;
    }
    static NodeSLL_2511533029 GetNode_3029(int data_3029) {
        return new NodeSLL_2511533029(data_3029);
    }
    static NodeSLL_2511533029 insertPos(NodeSLL_2511533029 headNode_3029, int position_3029, int value_3029) {
        NodeSLL_2511533029 head_3029 = headNode_3029;
        if (position_3029 < 1)
            System.out.print("Invalid position");
        if (position_3029 == 1) {
            NodeSLL_2511533029 new_node_3029 = new NodeSLL_2511533029(value_3029);
            new_node_3029.next_3029 = head_3029;
            return new_node_3029;
        } else {
            while (position_3029-- != 0) {
                if (position_3029 == 1) {
                    NodeSLL_2511533029 newNode_3029 = GetNode_3029(value_3029);
```

Kode Program 2.2

```java
                    newNode_3029.next_3029 = headNode_3029.next_3029;
                    headNode_3029.next_3029 = newNode_3029;
                    break;
                }
                headNode_3029 = headNode_3029.next_3029;
            }
            if (position_3029 != 1)
                System.out.print("Posisi di luar jangkauan");   }
        return head_3029;
    }
    public static void printList(NodeSLL_2511533029 head_3029) {
        NodeSLL_2511533029 curr_3029 = head_3029;
        while (curr_3029.next_3029 != null) {
            System.out.print(curr_3029.data_3029 + "-->");
            curr_3029 = curr_3029.next_3029;
        }
        if (curr_3029.next_3029 == null) {
            System.out.print(curr_3029.data_3029);
        }
        System.out.println();
    }
    public static void main(String[] args) {
        // buat linked list 2->3->5->6
        NodeSLL_2511533029 head_3029 = new NodeSLL_2511533029(2);
        head_3029.next_3029 = new NodeSLL_2511533029(3);
        head_3029.next_3029.next_3029 = new NodeSLL_2511533029(5);
        head_3029.next_3029.next_3029.next_3029 = new NodeSLL_2511533029(6);

        // cetak list asli
        System.out.print("Senarai berantai awal:  ");
        printList(head_3029);

        // tambahkan node baru di depan
        System.out.print("tambah 1 simpul di depan: ");
        int data_3029 = 1;
        head_3029 = insertAtFront(head_3029, data_3029);
        // cetak update list
        printList(head_3029);

        // tambahkan node baru di belakang
        System.out.print("tambah 1 simpul di belakang: ");
```

Kode Program 2.3

```java
        int data2_3029 = 7;
        head_3029 = insertAtEnd(head_3029, data2_3029);
        // cetak update list
        printList(head_3029);

        System.out.print("tambah 1 simpul ke data 4: ");
        int data3_3029 = 4;
        int pos_3029 = 4;
        head_3029 = insertPos(head_3029, pos_3029, data3_3029);
        // cetak update list
        printList(head_3029);
    }
}
```

Kode Program 2.4

Program TambahSLL_2511533029 merupakan implementasi operasi penambahan node pada struktur data Single Linked List. Program ini memiliki beberapa method utama yang digunakan untuk menambahkan node pada posisi yang berbeda. Method insertAtFront()

digunakan untuk menambahkan node di bagian depan linked list dengan cara membuat node baru, menghubungkannya ke head lama, lalu menjadikannya head yang baru. Method insertAtEnd() digunakan untuk menambahkan node di bagian akhir linked list dengan cara menelusuri node dari awal hingga node terakhir, kemudian menghubungkan node terakhir tersebut ke node baru. Selanjutnya, method insertPos() digunakan untuk menambahkan node pada posisi tertentu. Method ini bekerja dengan menelusuri linked list hingga posisi yang diinginkan, kemudian menyisipkan node baru di antara node sebelumnya dan node sesudahnya. Selain itu, terdapat method printList() yang berfungsi untuk menampilkan isi linked list dari awal hingga akhir dengan format panah (-->) sebagai penanda hubungan antar node.

Pada method main(), program terlebih dahulu membuat linked list awal dengan isi data 2-->3-->5-->6. Setelah itu, program menampilkan linked list awal, lalu melakukan penambahan node di depan dengan nilai 1, sehingga linked list menjadi 1-->2-->3-->5-->6. Selanjutnya program menambahkan node di belakang dengan nilai 7, sehingga linked list berubah menjadi 1-->2-->3-->5-->6-->7. Terakhir, program menambahkan node bernilai 4 pada posisi ke-4, sehingga hasil akhir linked list menjadi 1-->2-->3-->4-->5-->6-->7. Program ini menunjukkan bagaimana proses penambahan node pada Single Linked List dapat dilakukan di bagian depan, belakang, maupun posisi tertentu secara dinamis.

2.2.3 Class PencarianSLL_2511533029

Kode program pencarian single linked list merupakan kode program yang menjelaskan proses pencarian data pada Single Linked List dilakukan dengan cara menelusuri node satu per satu dari awal hingga akhir. Berikut kode program pencarian SLL:

```java
package pekan5_2511533029;

public class PencarianSLL_2511533029 {
    static boolean searchKey (NodeSLL_2511533029 head_3029, int key_3029) {
        NodeSLL_2511533029 curr = head_3029;
        while (curr != null) {
            if (curr.data_3029 == key_3029)
                return true;
            curr = curr.next_3029;
        }
            return false;
        }
    public static void traversal_3029 (NodeSLL_2511533029 head_3029) {
        //mulai dari head
        NodeSLL_2511533029 curr= head_3029;
        //telusuri sampai pointer null
        while (curr != null) {
            System.out.print(" " +curr.data_3029);
            curr = curr.next_3029; }
            System.out.println();
        }
            public static void main(String[] args) {
                NodeSLL_2511533029 head_3029 = new NodeSLL_2511533029 (14);
                head_3029.next_3029 = new NodeSLL_2511533029 (21);
                head_3029.next_3029.next_3029 = new NodeSLL_2511533029 (13);
                head_3029.next_3029.next_3029.next_3029 = new NodeSLL_2511533029 (30);
                head_3029.next_3029.next_3029.next_3029.next_3029 = new NodeSLL_2511533029 (10);
                System.out.print("penulusuran ssl : ");
                traversal_3029 (head_3029);
                // databyang akan dicari
                int key_3029 = 30;
                System.out.print("cari data " + key_3029 + " : ");
                if (searchKey(head_3029, key_3029))
                    System.out.println("ketemu");
                else {
                    System.out.println("tidak ada");
                }
            }
    }
```

Kode Program 2.5

Program PencarianSLL_2511533029 merupakan implementasi operasi pencarian data pada struktur data Single Linked List. Program ini digunakan untuk menelusuri isi linked list sekaligus mencari apakah suatu nilai tertentu terdapat di dalam list. Method searchKey() berfungsi untuk mencari data pada linked list. Proses pencarian dilakukan dengan menelusuri node mulai dari head hingga node terakhir menggunakan pointer curr. Setiap node diperiksa apakah nilai pada data_3029 sama dengan nilai yang dicari (key_3029). Jika data ditemukan, method akan mengembalikan nilai true, sedangkan jika seluruh node telah ditelusuri dan data tidak ditemukan, method akan mengembalikan nilai false.

Selain itu, program ini memiliki method traversal_3029() yang berfungsi untuk menampilkan seluruh isi linked list dari awal hingga akhir. Proses traversal dilakukan dengan menelusuri setiap node mulai dari head sampai pointer bernilai null, lalu mencetak nilai data pada

setiap node. Method ini digunakan agar pengguna dapat melihat isi linked list sebelum proses pencarian dilakukan.

Pada method main(), program terlebih dahulu membuat sebuah Single Linked List dengan data 14, 21, 13, 30, dan 10. Data tersebut dihubungkan satu per satu sehingga membentuk rangkaian linked list. Setelah itu, program menampilkan isi linked list menggunakan method traversal_3029(). Selanjutnya program menentukan nilai yang akan dicari, yaitu 30, lalu memanggil method searchKey() untuk melakukan pencarian. Jika nilai ditemukan, program akan menampilkan pesan "ketemu", sedangkan jika tidak ditemukan akan menampilkan pesan "tidak ada".

2.2.4 Class HapusSLL_2511533029

Kode Program terakhir untuk single linked list yaitu kode program yang menjelaskan bagaimana node pada Single Linked List dapat dihapus secara dinamis dari berbagai posisi, baik di awal, di akhir, maupun di posisi tertentu. Proses penghapusan dilakukan dengan mengatur ulang pointer antar node, sehingga struktur linked list tetap terhubung dengan baik setelah node dihapus. Berikut kode program hapus SLL:

```java
package pekan5_2511533029;

public class HapusSLL_2511533029 {

    // fungsi untuk menghapus head
    public static NodeSLL_2511533029 deleteHead_3029(NodeSLL_2511533029 head_3029) {
        // jika ssl kosong
        if (head_3029 == null)
            return null;

        // pindahkan head ke node berikutnya
        head_3029 = head_3029.next_3029;

        // return head baru
        return head_3029;
    }

    // fungsi menghapus node terakhir ssl
    public static NodeSLL_2511533029 removeLastNode_3029(NodeSLL_2511533029 head_3029) {
        // jika list kosong, return null
        if (head_3029 == null) {
            return null;
        }

        // jika list satu node, hapus node dan return null
        if (head_3029.next_3029 == null) {
            return null;
        }

        // temukan node terakhir kedua
        NodeSLL_2511533029 secondLast_3029 = head_3029;
        while (secondLast_3029.next_3029.next_3029 != null) {
            secondLast_3029 = secondLast_3029.next_3029;
        }

        // hapus node terakhir
        secondLast_3029.next_3029 = null;
        return head_3029;
    }
```

Kode Program 2.6

```java
    // fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511533029 deleteNode(NodeSLL_2511533029 head_3029, int position_3029) {
        NodeSLL_2511533029 temp_3029 = head_3029;
        NodeSLL_2511533029 prev_3029 = null;

        // jika linked list null
        if (temp_3029 == null) {
            return head_3029;
        }

        // kasus 1: hapus node pertama
        if (position_3029 == 1) {
            head_3029 = temp_3029.next_3029;
            return head_3029;
        }

        // kasus 2: menghapus node di tengah
        // telusuri ke node yang dihapus
        for (int i = 1; temp_3029 != null && i < position_3029; i++) {
            prev_3029 = temp_3029;
            temp_3029 = temp_3029.next_3029;
        }

        // jika ditemukan, hapus node
        if (temp_3029 != null) {
            prev_3029.next_3029 = temp_3029.next_3029;
        } else {
            System.out.println("Data tidak ada");
        }

        return head_3029;
    }

    // fungsi mencetak ssl
    public static void printList_3029(NodeSLL_2511533029 head_3029) {
        NodeSLL_2511533029 curr = head_3029;

        while (curr != null) {
            if (curr.next_3029 != null) {
                System.out.print(curr.data_3029 + "-->");
            } else {
```

Kode Program 2.7

```java
                System.out.print(curr.data_3029);
            }
            curr = curr.next_3029;
        }
        System.out.println();
    }
    // kelas main
    public static void main(String[] args) {
        // buat ssl 1->2->3->5->6->null
        NodeSLL_2511533029 head_3029 = new NodeSLL_2511533029(1);
        head_3029.next_3029 = new NodeSLL_2511533029(2);
        head_3029.next_3029.next_3029 = new NodeSLL_2511533029(3);
        head_3029.next_3029.next_3029.next_3029 = new NodeSLL_2511533029(5);
        head_3029.next_3029.next_3029.next_3029.next_3029 = new NodeSLL_2511533029(6);

        // cetak list awal
        System.out.print("List awal: ");
        printList_3029(head_3029);

        // hapus node terakhir
        head_3029 = removeLastNode_3029(head_3029);
        System.out.print("List setelah node terakhir dihapus: ");
        printList_3029(head_3029);

        // hapus node pada posisi 2
        int position_3029 = 2;
        head_3029 = deleteNode(head_3029, position_3029);

        // print list after deletion
        System.out.print("List setelah posisi 2 dihapus: ");
        printList_3029(head_3029);
    }
}
```

Kode Program 2.8

Program HapusSLL_2511533029 merupakan implementasi operasi penghapusan node pada struktur data Single Linked List. Program ini digunakan untuk menghapus node pada beberapa posisi, yaitu di bagian awal, di bagian akhir, dan pada posisi tertentu. Method deleteHead_3029() berfungsi untuk menghapus node pertama atau head dari linked list. Jika linked list kosong, method akan mengembalikan null. Namun jika terdapat node, maka head akan dipindahkan ke node berikutnya sehingga node pertama terhapus dari list. Method removeLastNode_3029() digunakan untuk menghapus node terakhir pada linked list. Prosesnya dilakukan dengan menelusuri linked list hingga menemukan node sebelum node terakhir, kemudian pointer node tersebut diubah menjadi null agar node terakhir terputus dari rangkaian.

Selain itu, program ini memiliki method deleteNode() yang berfungsi untuk menghapus node pada posisi tertentu. Method ini bekerja dengan menelusuri linked list hingga mencapai posisi node

yang akan dihapus. Jika posisi yang dipilih adalah posisi pertama, maka proses penghapusan dilakukan seperti pada penghapusan head. Namun jika node yang dihapus berada di tengah atau akhir, maka pointer dari node sebelumnya akan diarahkan langsung ke node setelah node yang dihapus, sehingga node tersebut terlepas dari linked list. Jika posisi yang dimasukkan tidak sesuai atau data tidak ditemukan, program akan menampilkan pesan bahwa data tidak ada.

Method printList_3029() digunakan untuk menampilkan isi linked list dari awal hingga akhir. Proses penelusuran dilakukan dari head sampai node terakhir, lalu setiap data dicetak dengan tanda panah (-->) untuk menunjukkan hubungan antar node. Method ini memudahkan pengguna untuk melihat perubahan isi linked list setelah proses penghapusan dilakukan.

Pada method main(), program terlebih dahulu membuat linked list awal dengan data 1-->2-->3-->5-->6. Setelah itu, program menampilkan isi linked list awal. Selanjutnya program menghapus node terakhir, sehingga linked list berubah menjadi 1-->2-->3-->5. Kemudian program menghapus node pada posisi ke-2, sehingga hasil akhir linked list menjadi 1-->3-->5. Setiap perubahan ditampilkan menggunakan method printList_3029() agar hasil penghapusan dapat terlihat dengan jelas.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Single Linked List merupakan salah satu struktur data linear yang tersusun dari kumpulan node yang saling terhubung melalui pointer. Setiap node menyimpan data dan alamat menuju node berikutnya, sehingga struktur ini bersifat dinamis dan memudahkan proses manipulasi data. Melalui praktikum ini, dapat dipahami bahwa Single Linked List dapat diimplementasikan dalam bahasa Java dengan beberapa operasi dasar seperti traversal, penambahan node, pencarian data, dan penghapusan node. Setiap operasi dilakukan dengan mengatur hubungan antar node melalui pointer, sehingga data dapat dikelola secara fleksibel tanpa harus bergantung pada ukuran tetap seperti array. Dengan demikian, praktikum ini membantu mahasiswa memahami konsep dasar Single Linked List serta penerapannya dalam pemrograman secara lebih terstruktur dan sistematis.

3.2 Saran

Saran yang dapat diberikan untuk praktikum selanjutnya adalah mahasiswa sebaiknya lebih teliti dalam memahami alur hubungan antar node, terutama saat mengatur pointer pada proses penambahan dan penghapusan data, karena kesalahan kecil dapat memengaruhi keseluruhan struktur linked list. Selain itu, mahasiswa disarankan untuk lebih sering berlatih membuat dan memodifikasi program Single Linked List agar semakin memahami logika kerja struktur data ini.

DAFTAR PUSTAKA

[1] Arbath Abdurrahman, “Struktur Data Single Linked List,”. [Daring]. Tersedia pada: Struktur Data Single Linked List [Diakses: 4-Mei-2026].

[2] Diana, “Linked list, single, double dan circular,”. [Daring]. Tersedia pada: Linked List: Single, Double, dan Circular Linked List [Diakses: 4-Mei-2026].

LAPORAN TUGAS PRAKTIKUM

Struktur Data

Pekan 4 (Queue)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Tugas Single Linked List

Topik : Implementasi Single Linked List

Tema : Simulasi Antrian Rumah Sakit

Deskripsi Tugas

Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO — First In, First Out).

JAWABAN

1. Kode Program

Berdasarkan intruksi tugas single linked list pekan 5, class yang akan dibuat terdapat dua class, yaitu class untuk pasien dan rumah sakit.

1.1 Class Pasien_2511533029

```java
package pekan5_2511533029;
public class Pasien_2511533029 {
    String NamaPasien_3029;
    String Penyakit_3029;
    int NomorAntrian_3029;
    Pasien_2511533029 next_3029;

    // Konstruktor
    public Pasien_2511533029(String nama_3029, String penyakit_3029, int NomorAntrian_3029) {
        this.NamaPasien_3029 = nama_3029;
        this.Penyakit_3029 = penyakit_3029;
        this.NomorAntrian_3029 = NomorAntrian_3029;
        this.next_3029 = null;
    }
    // Getter
    public String getNamaPasien_3029 () {
        return NamaPasien_3029;
    }
    public String getPenyakit_3029 () {
        return Penyakit_3029;
    }
    public int getNomorAntrian_3029 () {
        return NomorAntrian_3029;
    }
    public Pasien_2511533029 getNext_3029 () {
        return next_3029;
    }
    // Setter
    public void setNamaPasien_3029 (String namaPasien_3029) {
        this.NamaPasien_3029 = namaPasien_3029;
    }
    public void setPenyakit_3029 (String penyakit_3029) {
        this.Penyakit_3029 = penyakit_3029;
    }
    public void setNomorAntrian_3029 (int nomorAntrian_3029) {
        this.NomorAntrian_3029 = nomorAntrian_3029;
    }
    public void setnext_3029 (Pasien_2511533029 next_3029) {
        this.next_3029 = next_3029;
    }
}
```

Kode Program 1.1

Program Pasien_2511533029 merupakan class node yang digunakan untuk merepresentasikan data pasien pada sistem antrian rumah sakit berbasis Single Linked List. Class ini menyimpan informasi penting setiap pasien, yaitu NamaPasien_3029 untuk nama pasien, Penyakit_3029 untuk keluhan atau penyakit pasien, NomorAntrian_3029 untuk nomor urut antrian, serta next_3029 sebagai pointer yang menghubungkan node

pasien saat ini ke pasien berikutnya. Konstruktor pada class ini berfungsi untuk menginisialisasi seluruh data pasien saat objek dibuat, sekaligus mengatur nilai next_3029 menjadi null karena node baru belum terhubung ke node lain. Selain itu, class ini memiliki method getter untuk mengambil nilai atribut dan method setter untuk mengubah nilai atribut jika diperlukan. Secara keseluruhan, class ini berperan sebagai struktur dasar node dalam Single Linked List yang menyimpan data pasien dan menghubungkan setiap pasien dalam antrian secara berurutan.

1.2 Class RumahSakit_2511533029

```java
package pekan5_2511533029;

import java.util.Scanner;
public class RumahSakit_2511533029 {
    static Pasien_2511533029 head_3029 = null;
    static int urutan_3029 = 0;

    // Menambahkan di akhir
    public static void daftarkanPasien_3029 (String Nama_3029, String Penyakit_3029) {
        urutan_3029++;
        Pasien_2511533029 pasienBaru_3029 = new Pasien_2511533029 (Nama_3029, Penyakit_3029, urutan_3029);
        if (head_3029 == null) {
            head_3029 =  pasienBaru_3029;
            } else {
                Pasien_2511533029 temp_3029 = head_3029;
                while (temp_3029.next_3029 != null) {
                temp_3029 = temp_3029.next_3029;
            }
            temp_3029.next_3029 = pasienBaru_3029;
        }
        System.out.println("Pasien Berhasil Didaftarkan !!, Dengan Nomor Antrian: " + urutan_3029);

    }

    // Menghapus Antrian Terdepan
    public static void panggilPasien_3029 () {
        if (head_3029 == null) {
            System.out.println("Antrian Kosong, Tidak Ada Nomor Antrian");
            return;
        }

        System.out.println("===== Pasien dipanggil =====");
        System.out.println("Nomor Antrian    : " + head_3029.NomorAntrian_3029);
        System.out.println("Nama             : " + head_3029.NamaPasien_3029);
        System.out.println("Penyakit         : " + head_3029.Penyakit_3029);

        head_3029 = head_3029.next_3029;
    }

    // Display (Menampilkan Antrian)
    public static void tampilkanAntrian_3029 () {
```

Kode Program 1.2

```java
        if (head_3029 == null) {
            System.out.println("Antrian Kosong");
            return;
        }

        Pasien_2511533029 temp_3029 = head_3029;
        System.out.println("===== DAFTAR ANTRIAN PASIEN =====");
        while (temp_3029 != null) {
            System.out.println("Nomor Antrian   : " + temp_3029.NomorAntrian_3029);
            System.out.println("Nama Pasien     : " + temp_3029.NamaPasien_3029);
            System.out.println("Penyakit        : " + temp_3029.NomorAntrian_3029);
            System.out.println();
            temp_3029 = temp_3029.next_3029;
        }
    }

    // Mencari Pasien
    public static void cariPasien_3029 (String namaCari_3029) {
        Pasien_2511533029 temp_3029 = head_3029;

        while (temp_3029 != null) {
            if (temp_3029.NamaPasien_3029.equalsIgnoreCase(namaCari_3029)) {
                System.out.println("Pasien Ditemukan");
                System.out.println("Nomor Antrian   : " + temp_3029.NomorAntrian_3029);
                System.out.println("Nama Pasien     : " + temp_3029.NamaPasien_3029);
                System.out.println("Penyakit        : " + temp_3029.Penyakit_3029);
                System.out.println("");
                return;
            }
            temp_3029 = temp_3029.next_3029;
        }
        System.out.println("Pasien Tidak Ditemukan");
    }

    // Cek status
    public static void cekStatusAntrian_3029 () {
        if (head_3029 == null) {
            System.out.println("Antrian Kosong");
            return;
        }
```

Kode Program 1.3

```java
        int jumlah_3029 = 0;
        Pasien_2511533029 temp_3029 = head_3029;

        while (temp_3029 != null) {
            jumlah_3029++;
            temp_3029 = temp_3029.next_3029;
        }

        System.out.println("Jumlah pasien dalam antrian : " + jumlah_3029);
        System.out.println("Pasien terdepan             : " + head_3029.NamaPasien_3029);
    }
    public static void main (String [] args) {
        Scanner input_3029 = new Scanner (System.in);
        int pilihan_3029;

        do {
            System.out.println("\n=== Antrian Rumah Sakit NIM: 2511533029 ===");
            System.out.println("1. Daftarkan Pasien (Insert)");
            System.out.println("2. Panggil Pasien   (Delete Head)");
            System.out.println("3. Tampilkan Antrian (Display)");
            System.out.println("4. Cari Pasien      (Search)");
            System.out.println("5. Cek Status Antrian");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");
            pilihan_3029 = input_3029.nextInt();
            input_3029.nextLine();
            switch (pilihan_3029) {
            case 1:
                System.out.print("Masukkan Nama Pasien : ");
                String nama_3029 = input_3029.nextLine();
                System.out.print("Masukkan Penyakit    : ");
                String keluhan_3029 = input_3029.nextLine();
                daftarkanPasien_3029(nama_3029, keluhan_3029);
                break;

            case 2:
                panggilPasien_3029();
                break;

```

Kode Program 1.4

```java
            case 3:
                tampilkanAntrian_3029();
                break;

            case 4:
                System.out.print("Masukkan nama pasien yang dicari: ");
                String cari_3029 = input_3029.nextLine();
                cariPasien_3029(cari_3029);
                break;

            case 5:
                cekStatusAntrian_3029();
                break;

            case 6:
                System.out.println("Program selesai.");
                break;

            default:
                System.out.println("Pilihan tidak valid.");
        }
    } while (pilihan_3029 != 6);

    input_3029.close();
  }
}
```

Kode Program 1.5

Program RumahSakit_2511533029 merupakan implementasi struktur data **Single Linked List (SLL)** untuk mensimulasikan sistem antrian pasien di rumah sakit. Pada program ini, variabel head_3029 berfungsi sebagai penunjuk ke pasien pertama (antrian terdepan), sedangkan urutan_3029 digunakan untuk memberikan nomor antrian

secara otomatis setiap kali pasien baru didaftarkan. Method daftarkanPasien_3029() berfungsi menambahkan pasien ke akhir antrian (insert at end), sesuai prinsip antrian **FIFO (First In First Out)**. Jika antrian masih kosong, pasien baru langsung menjadi head, sedangkan jika antrian sudah berisi, program akan menelusuri node terakhir lalu menambahkan pasien baru di belakangnya. Method panggilPasien_3029() digunakan untuk memanggil pasien terdepan dengan menampilkan data pasien, lalu menghapusnya dari antrian menggunakan konsep **delete head**, yaitu memindahkan head ke node berikutnya.

Selain proses tambah dan hapus, program ini juga memiliki beberapa operasi pendukung. Method tampilkanAntrian_3029() digunakan untuk menampilkan seluruh daftar pasien yang sedang menunggu dengan cara menelusuri linked list dari head sampai node terakhir. Method cariPasien_3029() berfungsi mencari pasien berdasarkan nama dengan teknik traversal, lalu menampilkan informasi pasien jika ditemukan. Sementara itu, method cekStatusAntrian_3029() digunakan untuk menghitung jumlah pasien dalam antrian sekaligus menampilkan nama pasien yang berada di posisi terdepan. Pada method main(), program dijalankan menggunakan menu interaktif berbasis Scanner, sehingga pengguna dapat memilih fitur seperti mendaftarkan pasien, memanggil pasien, menampilkan antrian, mencari pasien, mengecek status antrian, hingga keluar dari program.

2. Output

Berikut beberapa output berdasarkan 2 class kode program, class pasien dan class rumah sakit.

```
=== Antrian Rumah Sakit NIM: 2511533029 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien   (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien      (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : Abdul
Masukkan Penyakit    : Demam
Pasien Berhasil Didaftarkan !!, Dengan Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511533029 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien   (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien      (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : Nabil
Masukkan Penyakit    : Gigi
Pasien Berhasil Didaftarkan !!, Dengan Nomor Antrian: 2
```

```
=== Antrian Rumah Sakit NIM: 2511533029 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien   (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien      (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
===== Pasien dipanggil =====
Nomor Antrian    : 1
Nama             : Abdul
Penyakit         : Demam
```

```
=== Antrian Rumah Sakit NIM: 2511533029 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien   (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien      (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
===== DAFTAR ANTRIAN PASIEN =====
Nomor Antrian    : 2
Nama Pasien      : Nabil
Penyakit         : 2


=== Antrian Rumah Sakit NIM: 2511533029 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien   (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien      (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan nama pasien yang dicari: NABIL
Pasien Ditemukan
Nomor Antrian    : 2
Nama Pasien      : Nabil
Penyakit         : Gigi
```

```
=== Antrian Rumah Sakit NIM: 2511533029 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien   (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien      (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah pasien dalam antrian : 1
Pasien terdepan             : Nabil

=== Antrian Rumah Sakit NIM: 2511533029 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien   (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien      (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Program selesai.
```